

X: Analyses de données textuelles

Guillaume Guex guillaume.guex@unil.ch

Les données textuelles

Il existe deux courants qui utilisent des méthodes mathématiques/informatiques appliquées aux **données textuelles** :

- L'**analyse de données textuelle (ADT)**, qui fait partie des sciences humaines, utilise fréquemment des méthodes computationnelles afin d'étudier l'information contenue dans des textes.
- Le **traitement automatique des langues (TAL, en anglais NLP)**, qui est un sous-domaine de l'intelligence artificielle, vise à créer des algorithmes automatisant des tâches variées sur les données textuelles.

Quelques exemples de tâches de NLP :

- L'**annotation du texte**.
- La **traduction**.
- La **génération de textes**.
- La **synthèse**.
- Le **topic modeling**.
- La **classification de documents**.
- La **recherche d'information**.

Les données textuelles : pré-traitements

Les données textuelles se trouvent fréquemment sous la forme d'une collection de textes appelée un **corpus**. Plusieurs pré-traitements sont possibles :

- La **tokenization** consiste à séparer notre texte en **tokens**, c'est-à-dire en unités linguistiques. Cela peut être des phrases, des (groupes de) mots, des syllabes ou des lettres.
- La **normalisation** consiste à rendre le texte uniforme. On peut enlever la ponctuation, les nombres, les caractères spéciaux, mettre le texte en minuscule, étendre les contractions, etc.
- Le **retrait des stopwords**, les stopwords sont des mots très courants qui ne possèdent que peu d'utilité sémantique (de, et, que, etc.). Selon l'analyse ou la méthode utilisée, on les enlève.
- La **lemmatisation** ou le **stemming** consistent à transformer toutes les **flexions** d'un mot en une unique forme. (le stemming coupe une partie du mot, la lemmatisation ramène tous les mots à leur racine).

Plusieurs outils existent (parfois basés sur des méthodes de ML) pour **annoter (to tag)** ou faire une **analyse syntaxique (to parse)** des tokens, généralement au niveau des (groupes de) mots.

- L'annotation en **catégorie grammaticale (Part Of Speech)** va définir si le mot est un sujet, verbe, adjectif, etc.
- L'annotation des **entités nommées (Named Entity Recognition)** va identifier dans le texte les personnes, lieux, entreprises, date, etc.
- L'**analyse syntaxique (parsing)** permet d'établir des schémas de relations syntaxiques entre les différents mots d'une phrase.

Les données textuelles, dans leur formes pré-traitées, ne sont pas lisible par un algorithme. Il existe plusieurs manières de les transformer :

- Le **bag-of-words**.
- L'**encodage one-hot**.
- Les **embeddings**.

Nous commencerons par voir les méthodes utilisant la représentation **bag-of-words**.

La représentation bag-of-words

La représentation **bag-of-words** consiste à diviser le corpus en **unités de contexte** (documents, chapitres, paragraphes), puis de construire la matrice **unité-terme**.

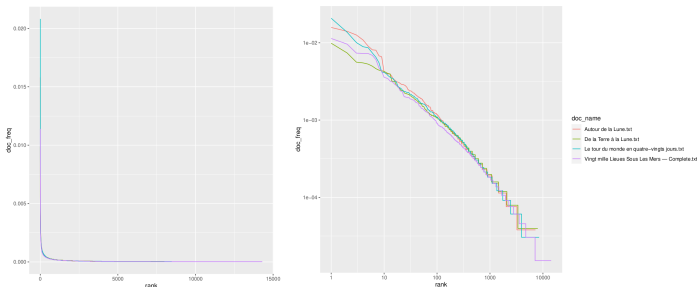
1 : « le chat mange le poisson » 2 : « le poisson est vieux » 3 : « mais un poisson est un poisson »

	chat	est	le	mais	mange	poisson	un	vieux
1	1	0	2	0	1	1	0	0
2	0	1	1	0	0	1	0	1
3	0	1	0	1	0	2	2	0

Les unités sont représentées par un **profil de fréquences**, mais l'**ordre des mots est perdu**.

La loi de Zipf

La somme des lignes donne la **fréquence des mots**, qui suit une **loi de Zipf** ($\sim$ une droite en échelle log-log) :



Les mots les plus fréquents sont certainement **peu spécifiques à des unités particulières**.

La pondération tf-idf

Une manière de mettre en valeur les mots les plus spécifiques à chaque document est d'appliquer une pondération de type **term frequency - inverse document frequency (tf-idf)**.

$$\text{tf-idf}(t, d) = \text{tf}(t, d) \cdot \text{idf}(t)$$

$$\text{tf}(t, d) = \frac{f_{t,d}}{\sum_{s \in d} f_{s,d}} \quad \text{idf}(t) = \log \left(\frac{n_C}{|\{d \in \mathcal{C} | t \in d\}|} \right)$$

où $f_{t,d}$ est la fréquence du terme t dans l'unité d , n_C le nombre d'unités dans le corpus $\mathcal{C}$, et $|\{d \in \mathcal{C} | t \in d\}|$ le nombre d'unités contenant t .

	chat	est	le	mais	mange	poisson	un	vieux
1	0.095	0	0.070	0	0.095	0	0	0
2	0	0.044	0.044	0	0	0	0	0.120
3	0	0.029	0	0.080	0	0	0.159	0

L'analyse des correspondances

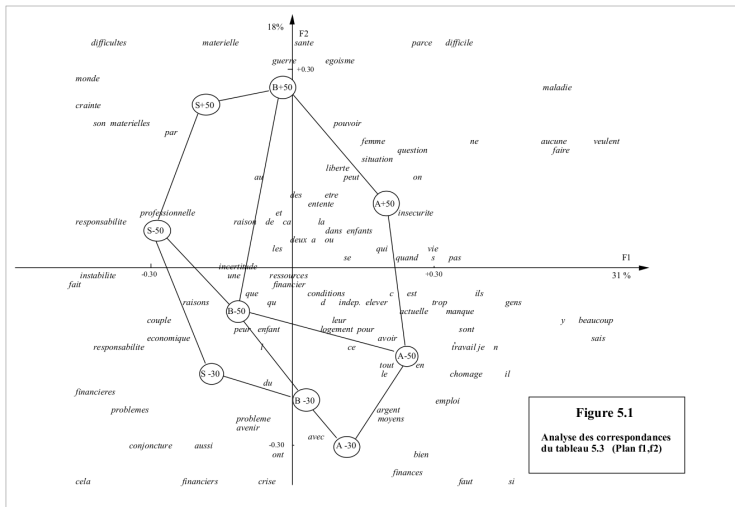
L'**analyse des correspondances (AFC)** est une méthode **factorielle** qui permet la représentation simultanée des **modalités de deux variables catégorielles**, représentées sous la forme d'une **table de contingence** (pour les détails, voir MQIII).

Elle peut donc s'appliquer sur une **table unité-terme**, permettant une représentation simultanée des **unités** et des **termes**, selon leur profil.

Depuis sa création par Benzécri en 1963, elle s'utilise comme **méthode exploratoire pour analyser les données textuelles**, en particulier dans le monde francophone.

L'analyse des correspondances

Le résultat principal de l'AFC est le **biplot**.



On peut l'interpréter avec les clés suivantes :

- Faire une AFC sur la matrice unité-terme correspond à faire un **MDS** sur les unités et sur les termes à partir de la **distance du chi2** entre leur profil de fréquence.
- Deux unités **proches** (resp. deux termes) auront des **profils similaires** du point de vue des termes (resp. des unités).
- Un **profil moyen** est représenté près du centre, un **profil atypique** en est éloigné.
- L'**attraction entre une unité et un terme** se mesure via le **produit scalaire** entre leur vecteur.
- Le **nombre d'axes factoriels** est de $\alpha = \min(n - 1, p - 1)$.

De plus, l'AFC permet d'obtenir d'autres quantités permettant d'affiner l'interprétation :

- Le **pourcentage d'inertie** représenté sur chaque axe.
- Les **contributions** des unités (ou des termes) à chaque axe, qui permettent d'interpréter les axes.
- Les **cosinus carrés** entre les unités (ou les termes) et chaque axe, qui permettent de savoir une unité (ou un terme) est bien représentée sur l'axe en question.

Le Topic Modeling

Le **Topic Modeling** correspond à la tâche d'**extraction automatique des thèmes principaux abordés dans une collection de documents**. Les méthodes principales sont :

- L'**analyse sémantique latente (LSA)**.
- L'**allocation de Dirichlet latente (LDA)**.
- La **factorisation en matrices non-négatives (NMF)**.
- Le **processus de Dirichlet hiérarchique (HDP)**.

Nous allons nous concentrer ici sur l'**allocation de Dirichlet latente (LDA)**.

L'allocation de Dirichlet latente (LDA)

L'**allocation de Dirichlet latente (LDA)** se base sur la table **document-terme** de taille $(p \times n)$ d'un corpus donné, et va supposer que cette dernière est le résultat du **modèle génératif aléatoire** suivant. On suppose que :

- Il existe **k thèmes**.
- Un document d , de taille p_d , est un générateur de thèmes tirés selon une **loi multinomiale** $Mult(p_d, \theta_{d1}, \dots, \theta_{dk})$.
- Chaque thème s , de taille q_s , est un générateur de mots tirés selon une **loi multinomiale** $Mult(q_s, \varphi_{s1}, \dots, \varphi_{sn})$.
- Chaque document de taille p_d est généré en tirant la taille des thèmes $q_1, \dots, q_k \sim Mult(p_d, \theta_{d1}, \dots, \theta_{dk})$, puis pour chaque thème q_s , en tirant la répartition des mots $f_1, \dots, f_n \sim Mult(q_s, \varphi_{s1}, \dots, \varphi_{sn})$.

L'allocation de Dirichlet latente (LDA)

Termes dans
les thèmes

	avion	manger	soir	chose
Thème 1	0.6	0.15	0.2	0.05
Thème 2	0	0.5	0.4	0.1

Thèmes dans
les documents

	Thème 1	Thème 2
Document 1	0.7	0.3
Document 2	0.05	0.95
Document 3	0.45	0.55

Génération

Document x Terme

	avion	manger	soir	chose
Document 1	9	1	2	0
Document 2	0	12	9	1
Document 3	3	4	3	0

L'allocation de Dirichlet latente (LDA)

Le but est donc d'**estimer les paramètres** $\theta_{d1}, \dots, \theta_{dk}$ et $\varphi_{s1}, \dots, \varphi_{sn}$ en fonction des données observées D , en maximisant :

$$\mathbb{P}(D | \theta_{d1}, \dots, \theta_{dk}, \varphi_{s1}, \dots, \varphi_{sn})$$

La LDA prend une approche bayésienne, en supposant que les paramètres suivent des lois **à priori** :

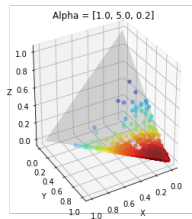
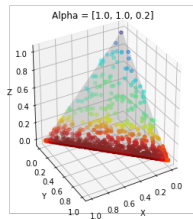
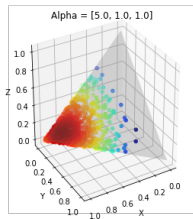
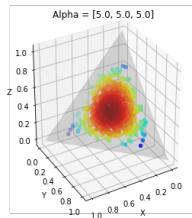
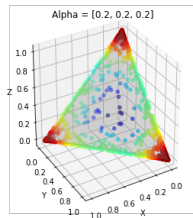
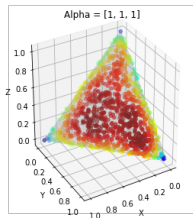
$$\theta_{d1}, \dots, \theta_{dk} \sim \text{Dir}(\alpha)$$

$$\varphi_{s1}, \dots, \varphi_{sn} \sim \text{Dir}(\eta)$$

où $\text{Dir}(\alpha)$ et $\text{Dir}(\eta)$ sont des **lois de Dirichlet** avec **hyperparamètres** α et η .

La loi de Dirichlet

En réalité, une loi de Dirichlet a autant de paramètres que de modalités, mais on suppose ici qu'ils sont **identiques**.



L'allocation de Dirichlet latente (LDA)

On va donc trouver $\theta_{d1}, \dots, \theta_{dk}$ et $\varphi_{s1}, \dots, \varphi_{sn}$ qui maximise :

$$\mathbb{P}(D|\theta_{d1}, \dots, \theta_{dk}, \varphi_{s1}, \dots, \varphi_{sn})\mathbb{P}(\theta_{d1}, \dots, \theta_{dk}|\alpha)\mathbb{P}(\varphi_{s1}, \dots, \varphi_{sn}|\eta)$$

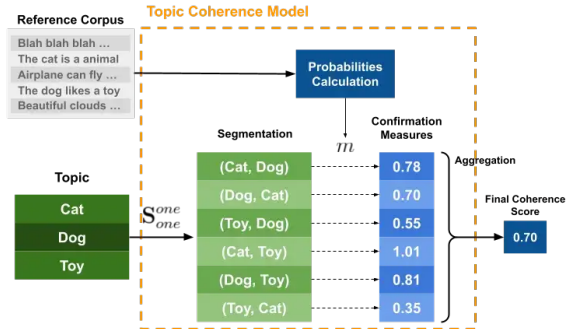
Que l'on peut résoudre via une **méthode bayésienne variationnelle** (une extension des méthodes EM).

Les **hyperparamètres** contrôlent la concentration des thèmes dans les documents et la concentration des mots dans les thèmes :

- Plus α est élevé, plus les documents contiennent de nombreux thèmes.
- Plus η est élevé, plus les thèmes contiennent de nombreux mots.

Les mesures de cohérence

Pour **évaluer une modèle avec les hyperparamètres k, α, η donnés**, on peut se baser sur de nombreuses **mesures de cohérence**. Ces dernières vont évaluer si le modèle peut générer un corpus similaire à un corpus de référence.



Source : <https://towardsdatascience.com/understanding-topic-coherence-measures-4aa41339634c>

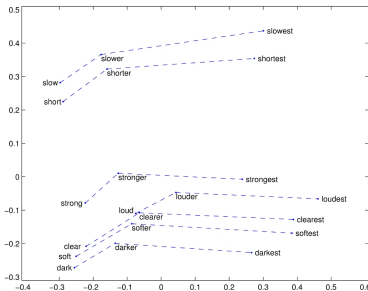
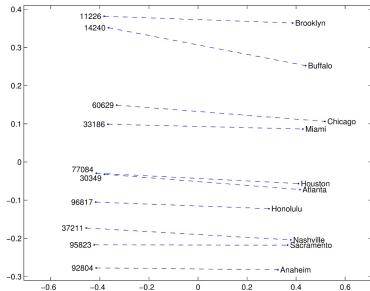
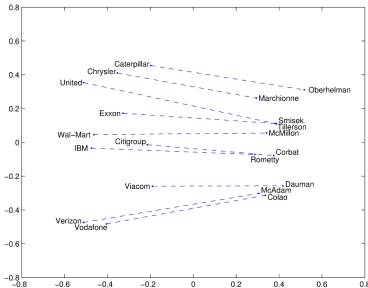
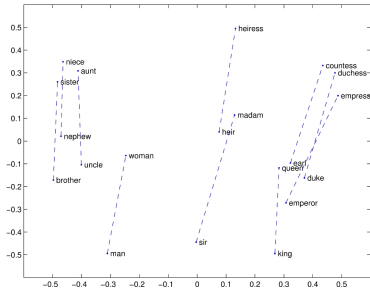
Les word embeddings

L'idée derrière les **word embeddings** est d'encoder les mots du vocabulaire d'un corpus par des **vecteurs qui contiennent de l'information sémantique ou syntaxique**. Les méthodes les plus connues sont :

- Word2Vec (Mikolov et al., 2013)
- GloVe (Pennington et al., 2014)
- Fasttext (Bojanowski et al., 2016)

Ces dernières sont des méthodes qui construisent des embeddings sur les **types**. Aujourd'hui, on utilise plutôt des LLM pour construire des word embeddings **contextuels**.

Les word embeddings



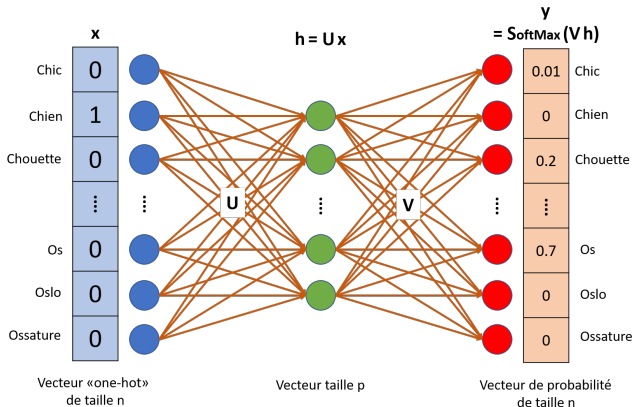
Source : <https://nlp.stanford.edu/projects/glove/>

Ces méthodes utilisent un grand corpus pour **entraîner les vecteurs**. Ces derniers vont être placés dans l'espace en se basant sur les **co-occurrences des mots dans un certain contexte**. Ce contexte peut être :

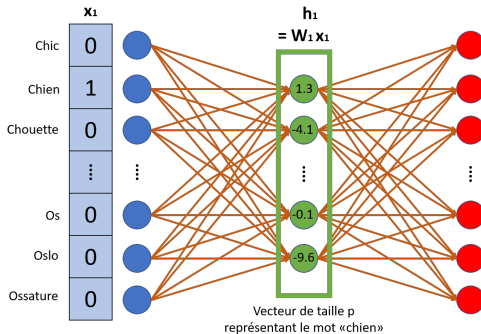
- Une **fenêtre coulissante** de taille définie.
- Une **phrase**.
- Un **paragraphe**.
- Un **document entier**.

Le plus fréquemment, ces méthodes utilisent une fenêtre de mots coulissante.

La méthode **Word2Vec** (version skip-gram) consiste à utiliser un **réseau de neurones** qui est entraîné sur une tâche « prétexte » : ce réseau doit **prédire le voisinage d'un mot**.



L'**entraînement** du réseau consiste à faire passer plusieurs **paires de mots [mot, mot du contexte]** comme entrée et sortie (avec un rééchantillonnage qui accentue les mots rares). Une fois le réseau entraîné, c'est la **couche intermédiaire** qui nous donnera les vecteurs représentant nos mots.



Les méthodes de Word Embedding peuvent être utilisées de deux manières différentes :

- Si l'on possède un corpus suffisamment grand, on peut entraîner les vecteurs à partir de celui-ci. On aura alors des vecteurs qui reflètent les **relations syntaxiques et sémantiques de notre corpus**.
- Il existe de base de données contenant des **vecteurs pré-entraînés**, généralement issus d'une très grand corpus. On peut alors **transformer un corpus donné en une séquence de vecteurs**, et utiliser différentes méthodes sur cette dernière.